

Interactive GPU-based Decimation of Large Meshes

Pascal Gautron
NVIDIA Corporation
USA
pgautron@nvidia.com

Christoph Kubisch
NVIDIA Corporation
USA
ckubisch@nvidia.com

Tristan Lorach
NVIDIA Corporation
USA
tlorach@nvidia.com

ABSTRACT

Remeshing and decimation algorithms are crucial for rendering levels of details on large geometry, as well as preparing meshes for the micro-mesh feature introduced with NVIDIA Ada generation. While remeshing is usually a time-consuming process running on a single or few CPU threads, we present a novel approach to the edge-collapse algorithm designed towards parallel, lock-free GPU decimation, allowing for interactive remeshing of large meshes.

CCS CONCEPTS

• Computing methodologies → Mesh geometry models.

ACM Reference Format:

Pascal Gautron, Christoph Kubisch, and Tristan Lorach. 2023. Interactive GPU-based Decimation of Large Meshes. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3595422>

1 EDGE COLLAPSE ALGORITHM

The Edge Collapse algorithm [Garland and Heckbert 1997] is a widely used iterative technique for mesh decimation, in which the edges of a triangular mesh are sorted according to the value of a cost function. At each step the edge with the lowest cost is collapsed by merging its extremities, and the edge costs are updated. While its simplicity makes this method a very robust solution for geometry simplification, the algorithm is inherently iterative. Parallelization of the edge collapse algorithm has been the subject of extensive research [DeCoro and Tatarchuk 2007], existing methods rely on parallel decimation of coarse clusters of the original mesh. The number of clusters is user-defined, and the behavior of the algorithm needs to be adjusted at the cluster edges. Furthermore, existing algorithms suppose the existence of topological information. Since triangular meshes are usually represented by vertex and index buffers, the topological information is generated upon loading the mesh, in an offline process not accounted for in the reported timings.

In contrast our algorithms below operate directly on the input triangles of a manifold mesh, building its topology and decimating in parallel. Our approach also preserves the discontinuities of vertex attributes, such as UV islands.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGGRAPH '23 Talks, August 06–10, 2023, Los Angeles, CA, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0143-6/23/08.
<https://doi.org/10.1145/3587421.3595422>



Figure 1: Lucy model decimated from 28M to 60K triangles in 15s while preserving finer features like the fingers.

2 BUILDING TOPOLOGY INFORMATION

2.1 Deduplication

We consider an input mesh consisting of a vertex buffer V_B and an index buffer I_B , the former containing the vertex attributes and the latter defining the triangles as triplets of vertex indices. Depending on the content creation software the vertex buffer may contain vertices whose attributes are identical. We use a concurrent hash map to identify such vertices and update the index buffer.

Some distinct vertices may share a same location, for example at texture coordinate discontinuities. Using a hash map based on the 3D vertex positions only we generate an additional index buffer P_B in which each referenced vertex has a unique position. We also flag the vertices corresponding to discontinuities.

2.2 Topology Lists

We extract connectivity information from the input triangle lists by building per-vertex linked lists identifying the triangles adjacent to each vertex. Let us consider a vertex v and define the index $v_h = \text{None}$ identifying the head of the list of triangles adjacent to v . For each triangle t in P_B we store a triplet t_h of indices representing the next entries in the adjacency list of each vertex of t .

We then consider each triangle t of P_B in parallel: for each vertex v of t we store its index v_h into t_h , and update v_h with the index of t . This operation can be done atomically, resulting in a linked list of triangles adjacent to each unique vertex location.

A list of unique edges is also built using a hash map, where the hash key for a given edge is based on the 3D locations of its vertices. Edges that are only referenced once in the hash map correspond to boundaries of the mesh, and are flagged accordingly. For each triangle we then store the indices of its edges.

3 SELECTING EDGES

The edges eligible for collapsing must satisfy criteria on the vertices, as well as on their associated cost. The topology building step above identifies vertices lying on discontinuities, either due to vertex attributes or mesh boundaries. In order to preserve those features the edges whose collapse would alter the topology of the discontinuities are not eligible for collapse (Figure 2). Furthermore, the cost function associated with the edges is compared to a user-defined threshold. Our cost function is based on a precomputed estimate of the surface curvature obtained by ray tracing, the edge length and the vertex valence resulting from the potential collapse. This cost function can be enriched with any other error estimate, for example based on quadrics [Garland and Heckbert 1997].

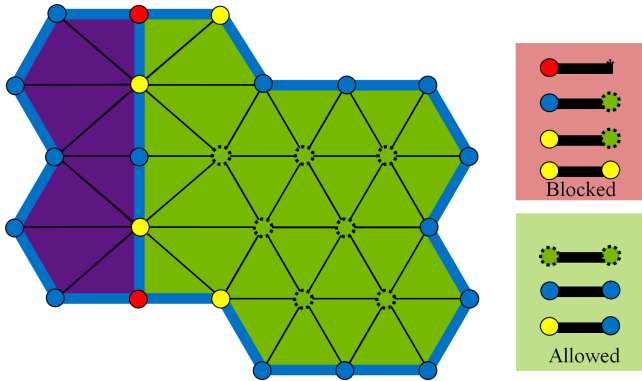


Figure 2: Eligible collapse candidates either lie along a discontinuity (in blue), or do not involve any discontinuity (green). Yellow vertices define edges adjacent to two distinct discontinuities. Discontinuity intersections (red) cannot be altered.

4 PARALLEL COLLAPSING

Collapsing an edge has side effects on the neighboring triangles, making the edge collapse algorithm inherently sequential. Attempts at parallelization partition the mesh into a number of smaller elements to guarantee collapse operations do not interfere with each other. We push this principle to the extreme by considering the very limited neighborhood affected by the collapse of any given edge. For an edge with index e_i and cost e_c we generate a 64-bit *edge descriptor* e_d containing e_c in the higher 32 bits and e_i in the remaining bits. We associate an edge descriptor t_d with each triangle, initialized to a default value with all bits set to 1. For each edge we iterate over the triangles adjacent to its vertices and atomically set $t_d = \min(t_d, e_d)$. As a result each minimal neighborhood containing an eligible edge only contains the descriptor of the edge with the smallest cost in its triangles.

For each edge we then consider its adjacent triangles, and check whether $t_d = e_d$ for each triangle. If yes the edge is collapsed by merging its vertices. As shown in Figure 3 this approach automatically maximizes the number of collapse operations performed in parallel while ensuring the absence of side effects. The process then restarts at the deduplication step (Section 2.1) until no edges satisfy the user-defined error criteria. The resulting vertex and index arrays are also compacted in-place on graphics hardware.

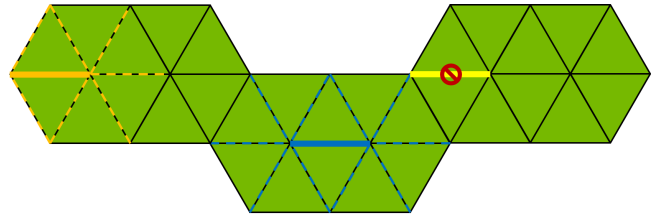


Figure 3: Using per-triangle *edge descriptors* only the direct neighborhood of the lowest-cost edge is blocked during an iteration, allowing other edges to be collapsed simultaneously.

5 GENERATING MICROMESH INFORMATION

The NVIDIA Ada generation introduces displacement micro-meshes, a very compact representation of scalar displacements stored on regular barycentric grids defined on the triangles of a lower-resolution mesh. By keeping track of edge collapse operations we analytically deduce the necessary subdivision levels and displacement bounds.

6 RESULTS AND FUTURE WORK

We implemented our algorithm using the Vulkan API running an NVIDIA GeForce 3090Ti. Simple models containing 2M triangles are decimated to 1% of their original triangle count in 1.4s (Figure 4). A larger model with 28M triangles is simplified to 112K triangles in 15s (Figure 1). The decimation speed also provides immediate feedback through interactive visualization of iterations.

The current solution requires a manifold input. Future work will consider adding capabilities to address T-junctions, vertex welding as well as objects containing several connected components.

REFERENCES

- Christopher DeCoro and Natalya Tatarchuk. 2007. Real-Time Mesh Simplification Using the GPU. In *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games* (Seattle, Washington) (I3D '07). Association for Computing Machinery, New York, NY, USA, 161–166. <https://doi.org/10.1145/1230100.1230128>
- Michael Garland and Paul S. Heckbert. 1997. Surface Simplification Using Quadric Error Metrics. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '97)*. ACM Press/Addison-Wesley Publishing Co., USA, 209–216. <https://doi.org/10.1145/258734.258849>

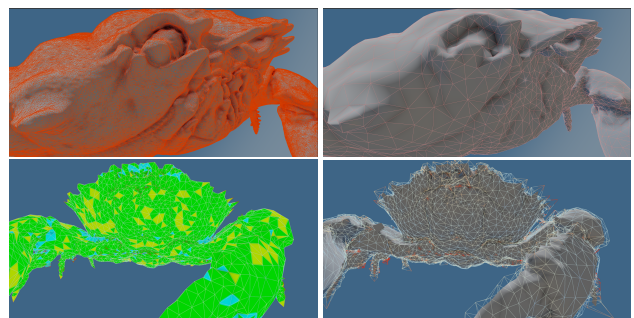


Figure 4: Reef Crab (2M triangles) decimated to 15K triangles in 1.4s (top). As a by-product the decimation process outputs the target decimation levels (bottom left, false colors) and the shell representing the displacement bounds (bottom right) for micromesh reconstruction.